

Fig. 2: Methodology for edge deployment of CSI-based joint activity recognition and localisation model

behaviours such as walking, sitting, and sleeping, allowing systems to automate lighting, temperature, and security based on user activity. While GPS is the conventional choice for localisation, it is unreliable indoors—an area where CSI excels.

In hospitals and assisted living facilities, CSI-based localisation helps monitor elderly patients or individuals with cognitive impairments, such as Alzheimer's. Caregivers can receive alerts if someone wanders into restricted zones or exits the premises. If a person collapses, the system can accurately detect their location and issue an emergency alert, reducing response time.

In smart spaces, the same activity may carry different meanings depending on context. A raised hand in the living room might signal increasing the TV volume, while the same gesture in the bedroom could adjust the AC. For this reason, recognising both activity and location simultaneously—known as CSI-based joint activity recognition and localisation—is essential.

Deep learning for CSI-based recognition

Once the CSI data is obtained, a model must be trained to identify real-time activity and location. A deep learning-based model, specifically a convolutional neural network (CNN), is employed for this task. Training a model involves mapping

the labelled input data to its output, during which the model learns parameters called weights and biases. These learned parameters are then used for real-time inference.

CNNs use filters to extract relevant information from

the data for the given task. These models, often comprising complex neural networks, require substantial memory and processing power, making real-time deployment on resource-constrained edge devices challenging. Deep neural networks used for CSI analysis frequently contain millions of parameters, rendering them unsuitable for embedded systems. Additionally, the limited processing power on edge devices increases inference latency, which impedes real-time application.

To address these challenges, the CNN-based model must be compressed in size and computational complexity for edge deployment. The model's size depends on the number of parameters (weights and biases), and its computational complexity depends on the number of floating-point operations per second (FLOPs). Model compression achieves this through two

primary techniques: quantisation and pruning.

Overcoming challenges via quantisation and pruning of CSI-based model

Quantisation reduces the numerical precision of model weights and activations. In neural networks, weights are typically stored as 32-bit floating-point (FP32) numbers. For example, if the value of a weight is 0.15625, in FP32 representation it is stored as 0 0111100 0100000000000000000000, where the first 0 represents the sign, the next 8 bits represent the exponent, and the last 23 bits represent the mantissa. FP32 representation takes 32 bits or 4 bytes. In order to reduce the precision, we perform quantisation. For INT8 quantisation, the values are mapped to a range having $2^8 = 256$ representable values. For INT8, this range can be used as $[-128, 127]$. Quantisation mathematically is given as:

$$x_q = \text{round}\left(\frac{1}{s} x - z\right)$$

where s is the scale factor, z is the value to which the original zero is mapped in lower precision, and the round function rounds off the value to the nearest integer. The value 0.15625 in INT8 precision will be represented as 20. INT8 precision takes only 1 byte compared to 4 bytes taken by FP32. So, by quantising the parameters to INT8 from FP32, we are reducing the model size by 4 times, which is a significant reduction. We can even quantise the weights to lower bit-widths. Obviously, this loss in precision introduces some quantisation error, which degrades the model accuracy, but we need to find an optimal balance between model size and performance.

Pruning, the other compression method, removes less important weights from the network, creating a

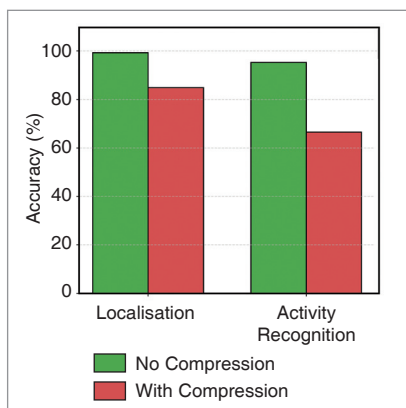


Fig. 3: Accuracy of the model with and without compression